Abstract

This portfolio project presents the conception, design, and implementation of a modular Habit Tracking Application developed in Python. The primary objective of the project is to demonstrate the ability to build a maintainable, testable, and extensible software system that adheres to clean architecture principles while providing meaningful functionality to end users. The application enables users to create, complete, track, and analyse habits with daily or weekly periodicity. Through the integration of a graphical user interface, persistent storage, and a dedicated analytics module, the project showcases a complete end-to-end software solution that aligns with the requirements of the IU portfolio assignment.

The system architecture is intentionally structured into distinct layers to ensure clarity, separation of concerns, and long-term maintainability. The user interface is implemented using Tkinter, providing a simple yet functional GUI that allows users to interact with the application intuitively. The logic layer, represented by the HabitManager, coordinates all operations between the UI, the domain model, the storage layer, and the analytics functions. The domain layer contains the Habit class, which encapsulates the essential properties and behaviours of a habit, including validation rules and streak calculations. The storage layer uses SQLite, an ACID-compliant relational database engine that ensures reliable persistence of habits and completion timestamps without requiring external services. The analytics layer is implemented using pure functions, making it easy to test, reason about, and extend.

A central focus of the project is the accurate calculation of habit streaks. Daily habits can be completed once per calendar day, while weekly habits follow ISO week numbering to ensure consistent behaviour across different calendar years. The application prevents double completion within the same period, ensuring that streaks remain meaningful and cannot be artificially inflated. The analytics module answers key questions such as listing all habits, filtering habits by periodicity, determining the longest streak overall, and calculating the longest streak for a selected habit. To support realistic analytics and robust testing, the project includes four weeks of predefined habit data, enabling time-series analysis and reproducible test scenarios.

Testing plays a crucial role in the development of the application. A comprehensive suite of unit tests validates the core functionality of the system, including habit creation, completion rules, deletion, streak logic, record streak detection, and all analytics functions. The test suite also covers important edge cases such as gaps in completion history, unsorted timestamps, empty datasets, and multiple habits with identical streak lengths. The use of pytest ensures that the tests are easy to run, maintain, and extend. The project follows Python best practices, including the use of docstrings, modular file organization, and a .gitignore file to avoid committing unnecessary files such as __pycache__ or temporary database files.

Beyond the technical implementation, the project reflects a deliberate focus on software quality, maintainability, and long-term extensibility. Throughout the development process, emphasis was placed on writing clean, readable code supported by meaningful documentation. The modular structure of the project allows individual components to evolve independently without introducing unnecessary coupling. For example, the current Tkinter interface could be replaced with a web-based or mobile interface, while the underlying logic

and analytics modules would remain unchanged. This demonstrates the flexibility and scalability of the chosen architecture.

From a learning perspective, the project provided valuable insights into the challenges of designing a complete software system from scratch. It required balancing simplicity with robustness, selecting appropriate technologies, and structuring the codebase in a way that supports both clarity and extensibility. The experience also reinforced the importance of documentation, version control, and consistent coding standards. Hosting the project on GitHub ensures transparency and allows others to review the code, run the application, and explore the test suite. This aligns with modern software development practices and demonstrates the ability to deliver a professional, well-structured project.

Looking ahead, the application offers numerous opportunities for enhancement. Potential future improvements include integrating data visualization to present streaks and habit trends more intuitively, adding user accounts to support multiple profiles, implementing cloud synchronization for cross-device access, and developing a mobile version to increase accessibility. Additional features such as reminders, notifications, or habit categories could further enrich the user experience. These extensions would build upon the solid architectural foundation established in this project and demonstrate how modular design enables continuous growth.

Overall, the Habit Tracking Application fulfills the requirements of the IU portfolio assignment and serves as a practical demonstration of effective software engineering principles. It showcases the ability to design a clean, modular architecture, implement reliable analytics, and validate functionality through comprehensive testing. The project stands as a complete, functional, and extensible solution that reflects both technical competence and thoughtful design.

GitHub Repository: https://github.com/elieassaf33/habit_app